

TP Concepteur développeur d'applications

Réussir mon titre Professionnel



ECF final



Objectif & Livrables

Application web, application mobile et application bureau

« Développer une application Web et mobile sécurisée et organisée en couches avec des outils de développement et environnement de tests »



Figure 1. page de garde du projet Cinéphoria

Préambule

Sommaire

Préambule	P.3
A – Introduction	P.3
B – Quelques définitions	P.4
C – Présentation de l'entreprise Natal Market et de Magento	P.4
• Présentation de Natal Market	P.5
• Présentation du logiciel Magento	P.6
I) Chapitre 1 – Le cahier des charges	P.7
• Mes principales missions	P.7
• Création des fiches pour chaque article	P.8
• Aperçu des résultats	P.9
II) Chapitre 2 – Le compte rendu d'activité	P.10
1. Gestion de la marchandise	P.10
2. Comment le site Web Natal Market attire plus de clients	P.10
• Le succès du travail en équipe	P.11
III) Chapitre 3 – Interprétation et critique des résultats	P.12
• Mécanisme et fonctionnement	P.12
• Des indicateurs clé avec Google Analytics	P.13
• Présence dans des catalogues et guides d'achats	P.14
V) Conclusion	P.15

Présentation de Cinéphoria

Cinéphoria est un joyau du cinéma français de la région toulousaine (dans le sud de la France), cette enseigne internationale avec 80 employés répartis dans 5 cinémas en France (Nantes, Bordeaux, Paris, Toulouse, Lille) et 2 en Belgique (Charleroi, Liège) veut créer un Web service

L'entreprise souhaite proposer à ses clients une plateforme moderne et performante afin d'améliorer leurs expériences car les employés sont assez lents dans l'accueil des clients, mais également avec l'outil informatisé. Les employés, qui doivent effectuer des opérations, n'ont pas d'accès à la gestion des salles et des séances, et de ce fait, toute la gestion se fait à la main

- **Application Web :**
 - ✦ Présenter les séances disponibles sur une heure donnée dans un cinéma donné
 - ✦ Visualiser tous les films projetés dans au moins un cinéma
 - ✦ Commander des billets pour une séance
 - ✦ Noter une séance de cinéma
- **Application mobile :**
 - ✦ Permettre aux clients de se connecter sur leurs comptes
 - Afficher le QR code d'un billet
- ✓ **Application bureautique :**
 1. Permettre aux employés de saisir des incidents dans une séance.

A retenir : l'Anglais est la langue officielle pour l'informatique, les graphiques etc.

Bâtir ce modèle de réussite en mode devOps

Les utilisateurs pourront consulter les films à l'affiche, réserver des billets, et les employés gérer les salles et les séances

Cinéphoria est une solution de qualité : analyse des besoins, maquettage, intégration, développement des règles de gestion, ainsi que le déploiement

Il est important de comprendre que notre client met à disposition des salles (entre 75-190 sièges), et leur nombre dépend de la ville et la fréquentation d'un film

A ce titre, idéalement au départ d'un projet : phase de développement logique, refactoring, design, des tests, version alpha, version bêta, référencement, lancement

Scénario

Cas de réalisation des tâches

Scénario : Tunnel de réservations.

Acteur : Clients

- |--- Consulter en navigant
- |--- Se connecter / s'inscrire
- |--- Consulter la liste des films
- |--- Consulter les séances
- |--- Réserver une séance et un nombre de siège
- |--- Télécharger ses billets
- |--- Annulation des billets

Acteur : Employés et Administrateur

- |--- Consulter en navigant
- |--- Se connecter / s'inscrire
- |--- Créer un film
- |--- Modifier un film
- |--- Supprimer un film
- |--- Consulter la liste des films
- |--- Supprimer des utilisateurs (Administrateur)

Base de données

Gestion des séquences

Chaque *action* de l'utilisateur passe par le **Contrôleur** (en reliant le **View** , au **Model**)

C'est PHP qui initie la *communication* avec PDO vers le système (No)SQL MySQL (MongoDB) par le port 3306 (de TCP/IP)

- ✓ Le front-Controller réalise les tâches par l'exécution de requêtes CRUD (où **GET** et **POST** sont des **méthodes Http(s)**)



Le schéma relationnel →

id	nom	prenom	user	password	email	date	role
----	-----	--------	------	----------	-------	------	------

Le schéma conceptuel →

[Client] 1 ---- N [Billet] N ---- 1 [Séance]

Serveur Apache

Sécurité

Apache comme WAMP ne dispose **pas de sécurité par défaut**

Or, c'est hyper important de sécuriser sa base de données stockée sur un PC serveur

2. Le pare-feu pour Apache doit s'installer (mod_security)

Pour cela on fait appel à **la cryptographie** (des protocoles **TLS, HTTPS**) et le **chiffrement des données via les mathématiques**. La question n'est pas de savoir comment mais si jamais notre système est attaqué quelle sera la riposte prévue en amont

3. La discrétion contribue au bien-être de tous

Les IP dynamiques sont gérées par le FAI Wifi du réseau

Nos données

PHP pur mais proche **de Laravel**, par sa sécurité, son architecture MVC, son routing

Les Sessions

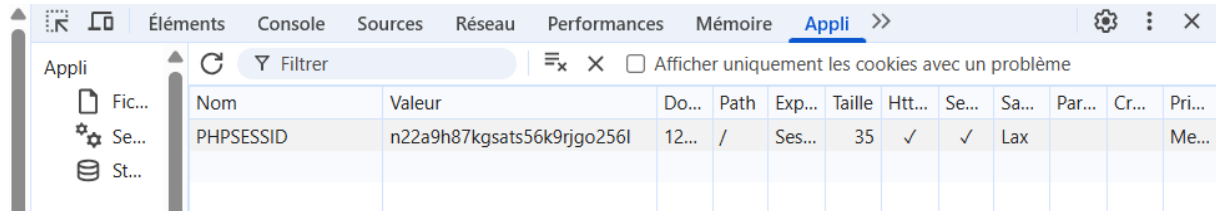
On n'utilisera **pas de jeton** « token » (JWT) dans ce projet par manque de temps et d'anticipation (**c'est un format JSON que je maîtrise mal**)

Les données de session sont stockées **côté serveur** (base de données), donc plus sécurisées que les cookies (côté client) avec des préférences de l'utilisateur

Cookies

4. Par défaut, un cookie sécurisé est un témoin de navigation PHP ; **mais il peut être collecté ou volé** 😬

Cookie PHPSESSID



Nom	Valeur	Do...	Path	Exp...	Taille	Htt...	Se...	Sa...	Par...	Cr...	Pri...
PHPSESSID	n22a9h87kgsats56k9rjgo256l	12...	/	Ses...	35	✓	✓	Lax			Me...

Sur Cinéphoria , on sécurise toutes les données du cookie via **HMAC_SECRET** avec les critères comme stipulés dans **nos Règlementations RGD**

- ✓ On peut **accepter ou refuser** le cookie « cinephoria » :

Cookie

Cookie Value ☒ Afficher les valeurs décodées via l'URL

```
{ "browser": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36", "date": "2025-09-29 15:13:46", "visits": 1, "ip": "127.0.0.1", "user": "client", "langue": "fr" }
```

Cookie avec HMAC_SECRET

Cookie Value ☒ Afficher les valeurs décodées via l'URL

```
eyJicm93c2VyljoiTW96aWxsYVwvNS4wLChXaW5kb3dzIE5UIDEwLjA7IFdpbjY0OyB4NjQpIEFwcGxlV2ViS2l0XC81MzcuMzYgK  
EtIVE1MLCBsaWtllEdiY2tvKSBdAHJvbWVcLzE0MC4wLjAuMCMBCBTYWZhcmlCLzUzNy4zNilslmRhGUiOilyMDI1LTA5LTl5IDE1OjI  
yOjI1liwidmlzaXRzljoxLCJpcCI6IjEyNy4wLjAuMSIsInVzZXliOiIjbjGllbnQlLCJsYW5ndWUiOiIjmcjI9.b919feb6689150b668e7ba0  
1643a5e6388a3e6e8df5278784fcf256e59058e81
```

Si le client accepte le cookie, alors on collectera :

```
$tableau = [
    'browser' => $browser,
    'date'    => $date,
    'visits'  => $visits,
    'ip'      => $ip,
    'user'    => $user,
    'langue'  => 'fr',
    'country' => $geo['country'],
    'city'    => $geo['city'],
```

Nous utilisons des cookies pour améliorer votre expérience. Acceptez-vous l'utilisation de cookies ?

Accepter

Refuser

```
'isp'      => $geo['isp'],
'latitude' => $geo['lat'],
'lon'      => $geo['lon']
];
```

Si le client refuse le cookie, alors on collectera :

```
$tableau = [
    'browser' => $browser,
    'date'    => $date,
    'visits'  => $visits,
    'ip'      => $ip,
    'user'    => $user,
    'langue'  => 'fr'
];
```

Les classes

Principe du **separation of concerns**, c'est plus facile de gérer la séparation entre **configuration** et **logique applicative**. C'est conforme aux bonnes pratiques de design pattern qui centralise les modules de notre application

- ✓ Des classes seront utilisées pour le process
- ✓ Travail en équipe

Le Modèle MVC

Le formulaire doit transmettre la demande de requête par le Controller à la base de données, qui elle sera stockée sur nos serveurs et donc inaccessible à distance

```
<!-- LoginController.php -->

<?php
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }

    $ip_path = '/src/View/components/IP.php';
    $email = isset($_POST['email']) ? trim($_POST['email']) : null;

    include_once ROOT_PATH . $ip_path;

    // Définit nos variables messages clairs pour la gestion des
```



```
erreurs/succès
$member = "Erreur Connexion $email : les données ne correspondent
pas à un membre ! IP : " . getUserIP() ;
$success_page = "confirmation-login";
$login_page = "login";
$catch_message = "Erreur Connexion $email : Problème ";

try {
    /* On essaie de se connecter en base */
    $conn = new
PDO("mysql:host={$_ENV['DB_HOST']};dbname={$_ENV['DB_NAME']};charset=utf8",
$_ENV['DB_USER'], $_ENV['DB_PASS']);
    // On veut définir le mode d'erreur de PDO sur Exception
    $conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

    if (isset($_POST['email']) && isset($_POST['password']) &&
!empty($_POST['email']) && !empty($_POST['password']))
    {
        $password = isset($_POST['password']) ?
trim($_POST['password']) : null;

        // demande la correspondance e-mail
        $sth = $conn->prepare($sql);
        $sth->execute([':email' => $email]);

        $resultat = $sth->fetch(PDO::FETCH_ASSOC);

        /* cas avec succès d'un tel e-mail */

        // Le résultat est différent par un "salt" aléatoire
        // password_verify (mot de passe clair // haché)
        : if (isset($resultat['id']) && password_verify($password,
$resultat['password']))
        {
            // rôles valides
            $_SESSION['user'] = $resultat['user'] ??
';

            $_SESSION['id'] = $resultat['id'];

            if ($resultat['role'] === 'employe')
            {
                $_SESSION['role'] = "employe";
            }
            else if ($resultat['role'] === 'client')
            {
                $_SESSION['role'] = "client";
            }
            else if ($resultat['role'] === 'admin')
```

```

        {
            $_SESSION['role'] = "admin";
        }
        header("Location: index.php?page=$success_page");
        exit();
    }
    else
    {
        error_log($member);
        $_SESSION['error']=true;

        header("Location: index.php?page=$login_page");
        exit();
    }
}

}

catch(PDOException $e){
    error_log($catch_message . $e->getMessage());
}

```

L'utilisateur est repéré par son identifiant unique (**id**)

La base de données

On doit utiliser des bases de données ?

En effet oui, **le principe du separation of concerns est clairement mis en lumière**

Ici dans ce cas :

- Pour le billet commandé du client
- Pour la gestion des utilisateurs , les usagers clients

Pourquoi ça ? Parce que personne hormis l'administrateur de la base de données ne pourra modifier ces informations, donc pas par les employés.

Cette option est bien sûr absente de l'interface pour tous les utilisateurs du service : de plus le serveur n'est pas trop surchargé.

C'est une mesure de sécurité importante qui est valide.

A l'inverse, pour la gestion des films on doit se servir du stockage, et appeler le serveur fréquemment car presque toujours un film ajouté peut contenir des erreurs de saisie de la part de l'individu "employé" (**role = "employé"**).

Finalement :

- **Table commandes** est reliée à un billet
- **Table membres** est reliée à un utilisateur

Pour cela on va se servir du MVC et plus particulièrement de la fonctionnalité du Controller.

La Technique

Techniquement, on utilise la classe moderne, universelle et sécurisée **PDO** pour **MySQL** (SQL) mais pour **MongoDB** (NoSQL) on utilisera **MongoDB\Client**. Ce sont ces classes qui abstraient les pilotes (mysqli / mongodb) natifs de bases de données, donc il faut définir les bons paramètres

✓ Pour PHP :

```
new PDO("mysql:host={$_ENV['DB_HOST']};dbname={$_ENV['DB_NAME']};charset=utf8", $_ENV['DB_USER'], $_ENV['DB_PASS']) ;
```

1. On utilisera des requêtes préparées pour y accéder

Rappel : c'est hyper important de **sécuriser** sa base de données stockée **côté serveur**

2. Le password sera haché en base par PHP avec l'**algorithme** B-CRYPT

```
// l'algorithme BCRYPT (hachage + salt) contient un mot de passe de 60 octets
$hashedPassword = password_hash($password, PASSWORD_BCRYPT);
```

3. **htmlspecialchars()** : le navigateur affiche le texte **brut** au lieu d'exécuter le script (fonction)

✓ **htmlspecialchars(\$userInput, ENT_QUOTES, 'UTF-8');**

4. **Utf-8** : ici utf-8 est un paramètre htmlspecialchars, c'est un encodage d'octets, pas une pratique de sécurité. Sans encodage utf-8, filtres de **sécurité** évités

- Pour MySQL :

- Notre Base de données et ses instructions relatives SQL (*cinephoria.sql*) :

```
CREATE DATABASE cinephoria;

DROP TABLE IF EXISTS `membres`;
CREATE TABLE IF NOT EXISTS `membres` (
```

```
`id` BIGINT NOT NULL AUTO_INCREMENT,
`nom` varchar(20) DEFAULT NULL CHECK (LENGTH(`nom`) > 3 AND LENGTH(`nom`) <
20),
`prenom` varchar(20) DEFAULT NULL CHECK (LENGTH(`prenom`) > 2 AND
LENGTH(`prenom`) < 20),
`user` varchar(20) DEFAULT NULL CHECK (LENGTH(`user`) > 8 AND LENGTH(`user`)
< 20),
`password` varchar(60) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci
DEFAULT NULL CHECK (`password` > 8 AND `password` < 20), -- codé et haché sur
60 caractères
`email` varchar(254) NOT NULL UNIQUE, -- email comme login
`date` date NOT NULL CHECK (date <= CURRENT_DATE),
`role` varchar(7) NOT NULL DEFAULT 'client' CHECK (`role` IN ('client',
'employe', 'admin')),
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
COMMIT;
```

- Nous avons défini des rôles : admin, employe, et client (par défaut)
- Le password (mot de passe) doit contenir exactement 60 caractères, 60 octets (1 c = 1 o) et avec l'encodage en UTF-8 des caractères spéciaux

✓ Pour MongoDB (en CLI) :

Ouvrir cmd → mongod → mongosh "mongodb://localhost:27017/cinephoria";
→ db.membres.insertMany([données à insérer json]) ; → db.membres.find().pretty() ;

Une fois MongoDB et ses outils binaires installés, **comment créer des données ?**
En fait, MongoDB ne stocke pas ses données comme un simple fichier qu'on ouvre, il stocke ses données dans un **data directory** géré par son **service**

- seul un utilisateur admin/root peut exécuter des commandes :

```
db.createUser({
  user: "admin",
  pwd: "root",
  roles: [{ role: "readWrite", db: "cinephoria" }]
});
```

- Accès IP limité (whitelist IP / localement) :

sur Windows :

Ouvre le Pare-feu Windows.

Crée une règle entrante :

Programme : mongod.exe

Port : 27017 (par défaut)

Autoriser seulement certaines IP Sauvegarde la règle.

- Écouter uniquement sur localhost (mongod.cfg)

```
# network interfaces
net:
  port: 27017
  bindIp: 127.0.0.1
```

- J'utilise le **pare-feu de Windows** (localement)
- autorisations et rôles
- chiffrement réseau

CODE ???

Menu Utilisateur

Structure HTML/PHP pour le menu de l'utilisateur :

Menu Employé

Le menu employé permet d'ajouter des films en ligne. De plus, voici l'interface :

Capture pièce-jointe.

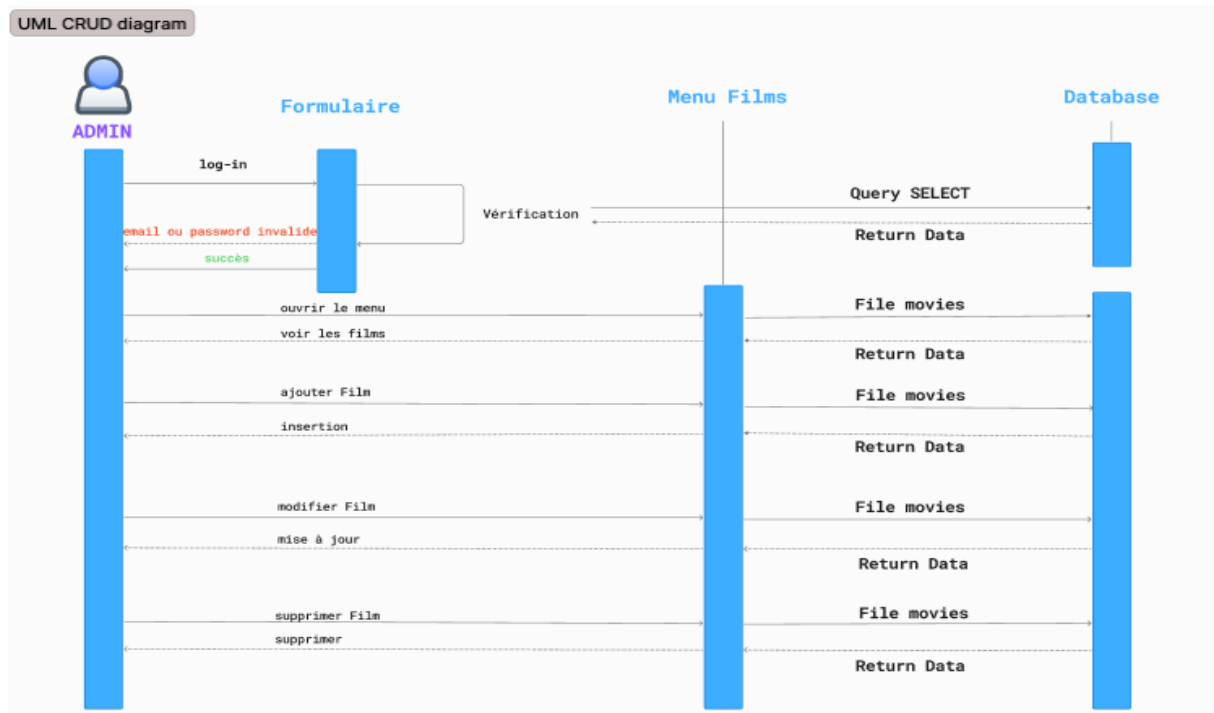
Structure HTML/PHP pour le menu de l'employé :

- analyser les fichiers joints img pochette
-

Menu Administrateur

Nous présenterons le menu Administrateur (rôle **admin**) car c'est là où l'on retrouve presque tous nos composants en un seul endroit, que je vais présenter maintenant

Structure HTML/PHP pour le menu d'administration



L'inscription / Le Log-in

Nous allons utiliser une adresse e-mail pour login et un nom d'utilisateur qui sera affiché dans l'application (admin@cinephoria.com , admin-123)

La gestion des films

La technique qui fait **le lien entre une base de données relationnelle** (SQL : tables, lignes, colonnes) et **le code orienté objet** (classes, objets, propriétés, méthodes) s'appelle ORM (**Object-Relational Mapping**)

Le nom de l'ORM dépend de la technologie (PHP pur,Laravel,Python,Node.js,Django pour Python, Spring Boot pour Java ...) mise en place par le serveur

- ✓ Ici , projet simple / e-learning : donc d'où pas d'ORM
- Pour PHP :
 1. Les requêtes sont exécutées par des méthodes : prepare(), execute(), fetch()... pour MySQL . MongoDB a ses propres méthodes .
 2. Des **objets PDOStatement** qui représentent **le résultat** des requêtes (PDO::FETCH_ASSOC, PDO::ERRMODE_EXCEPTION, etc.)

MongoDB utilise le client (**pilote mongodb**) pour communiquer avec le système de base de données MongoDB et se connecter à la base de données . C'est **une bibliothèque logicielle** qui traduit les appels fais dans ton code en requêtes que **MongoDB peut comprendre**

- Pour MongoDB :

???

Pour ajouter un film, on écrit d'abord le code suivant :

```
<!-- manage-products-handler.php -->
<!-- Gestion pour l'ajout ou modifier un film (movies.php) -->

<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();}
include_once ROOT_PATH . $movies_data_path;

// libre à vous de rajouter des conditions
function date_fr(){
    $date = new DateTime();
    $formatter = new IntlDateFormatter(
        'fr_FR',
        IntlDateFormatter::FULL,
        IntlDateFormatter::NONE,
        'Europe/Paris',
        IntlDateFormatter::GREGORIAN,
        "EEEE d MMMM y"
    );
    return ucfirst($formatter->format($date));
}

try {
/* Connexion à notre DataFile PHP.
Vérification si le formulaire est soumis,
si un champ est NULL, on affiche un message d'erreur dans le log. */

if (($_SERVER['REQUEST_METHOD'] == 'POST') &&
isset($_POST['titre'], $_POST['description'], $_POST['auteur'], $_POST['duree'], $_FILES['pochette'], $_POST['movie_release_date'], $_POST['cinema']) &&
!empty($_POST['titre']) && !empty($_POST['description']) &&
!empty($_POST['auteur']) && !empty($_POST['duree']) &&
($_FILES['pochette']['error'] === 0) && !empty($_POST['movie_release_date'])
&& !empty($_POST['cinema'])) {

    // Récupération des données du formulaire
    $title = $_POST['titre'];
    $subtitle = $_POST['subtitle'] ?? "null";
    $description = $_POST['description'];
    $movie_release_date = $_POST['movie_release_date'];
```

```

$duree = $_POST['duree'];
$auteur = $_POST['auteur'];
$pochette = $_FILES['pochette'];
$forbidden = $_POST['age'];
$version = $_POST['version'];
$genre = $_POST['genre'];

/* On sépare la logique ici */

// Traitement de l'image (pochette)
if ($_FILES['pochette']['error'] == 0) {

    // On définit le répertoire où stocker les images
    $uploadDir = 'uploads/';

    // Générer un nom unique
    $fileName = basename($_FILES['pochette']['name']);
    $ext = pathinfo($fileName, PATHINFO_EXTENSION);
    $newFileName = uniqid('film_', true) . "." . $ext;

    // Déplacer le fichier uploadé
    $uploadFile = $uploadDir . $newFileName;

    // Vérification de l'extension de l'image
    $allowedExtensions = ['jpg', 'jpeg', 'png', 'gif', 'webp', 'avif',
'heic', 'svg'];
    $allowedMimeTypes =
['image/jpg','image/jpeg','image/png','image/gif','image/webp','image/avif','i
mage/heic','image/svg'];

    $fileExtension = strtolower(pathinfo($uploadFile,
PATHINFO_EXTENSION));
    // Vérification taille
    $maxSize = 2 * 1024 * 1024; // 2 Mo

    $fileTmp = $_FILES['pochette']['tmp_name'];
    $fileExt = strtolower(pathinfo($fileName, PATHINFO_EXTENSION));
    $fileMime = mime_content_type($fileTmp);

    if (!in_array($fileMime, $allowedMimeTypes)) {
        die('Type de fichier non autorisé.');
```



```

        die('L'image dépasse la taille maximum (2Mo)');
    }

    // Déplacer l'image vers le dossier uploads
    else if (move_uploaded_file($fileTmp, $uploadFile)) {

        // Gérer le nouveau contenu PHP
        // Réécrire le DataFile avec le nouveau tableau
        // Nouvelle ligne à ajouter

        $nouvelleLigne = ["id" => count($films) + 1, "titre" =>
$_POST['titre'], 'subtitle' => $_POST['subtitle'], "description" =>
$_POST['description'], "auteur" => $_POST['auteur'], "duree" =>
$_POST['duree'], 'version' => $version, 'interdit' => $forbidden, 'genre' =>
$genre, "created_at" => date_fr(), "date_de_sortie" =>
date_movie_release($movie_release_date), "pochette" => $uploadFile];

        // Ajouter la ligne au tableau 2D
        $films[] = $nouvelleLigne;

        // Convertir en code PHP
        $contenu = "<?php\n\n$films = " . var_export($films, true) .
";\n";
    }
}

```

Serveur Apache

```

        // Réécrire le DataFile nouvelle version
        file_put_contents('../data/movies.php', $contenu);
    }

    catch (PDOException $e) {
        error_log("Erreur de connexion (gestion des produits) : " . $e-
>getMessage());
    }

    finally {
        header('Location: manage-products.php');
    }
}
?>

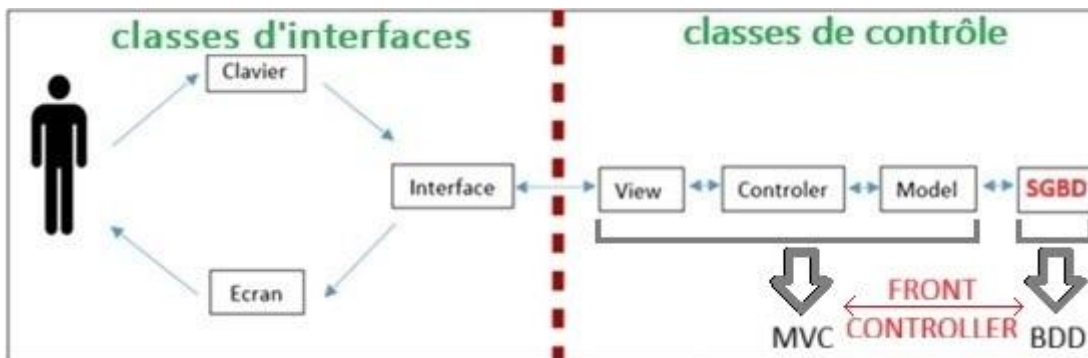
```

Les Tests

Pour PHP, le standard est **PHPUnit** pour les tests unitaires et fonctionnels

- pour le login au web service PHP
- pour l'inscription au web service PHP (3 rôles = 3 cas)
- pour l'ajout d'un film de l'employé (1 cas)
- pour l'entonnoir de vente utilisateur (1 cas)

```
Cinéphoria/
|— config/
|   └─ config.php      → Paramètres globaux (connexion BDD, constantes...)
|
|— public/
|   └─ index.php       → Point d'entrée (routeur / front controller)
|
|— src/
|   └─ Controller/     → Contrôleurs (logique de navigation)
|   └─ Model/          → Modèles (accès aux données, logique métier)
|   └─ View/           → Vues (HTML, templates, affichage)
|
|— vendor/             → Dépendances installées (via Composer)
|
|— .env                → Variables d'environnement (ex: accès DB)
|
|— composer.json       → Dépendances et autoloading
```



Menu Administrateur

1. Entrez le lien de votre outil de gestion de projet [JIRA](#)

NON

2. Entrez le lien de votre git [GIT](https://github.com/dimitribenhamma/Cinephoria/tree/develop)

<https://github.com/dimitribenhamma/Cinephoria/tree/develop>

Entrez le lien de vos applications déployées en ligne

NON

Partie 2 : Planification

1. Comment avez-vous effectué la planification de votre projet ?

Sur papier, puis sur ordinateur

2. Quelle méthode de gestion de projet avez-vous utilisé et pourquoi ?

J'ai choisi la méthode Kanban car mon rythme a été peu soutenu (3 semaines pour réaliser cet ECF). De plus, je suis seul pour réaliser ce travail.

Partie 3 : Technologies utilisées

1. Quelles technologies avez-vous utilisé ? Précisez tous les éléments qui vous ont permis de faire ce choix. Ajoutez également un paragraphe expliquant en quoi ces choix sont les plus adaptés pour la demande spécifiée dans l'énoncé.

Technologies : PHP JAVASCRIPT MYSQL HTML / CSS (Bootstrap)

2. Quelles mesures avez-vous prise afin de protéger vos applications des potentielles vulnérabilités de sécurité ? (Injection SQL, failles XSS, ...)

Injection SQL par des paramètres nommés

Glossaire

CRUD

MVC

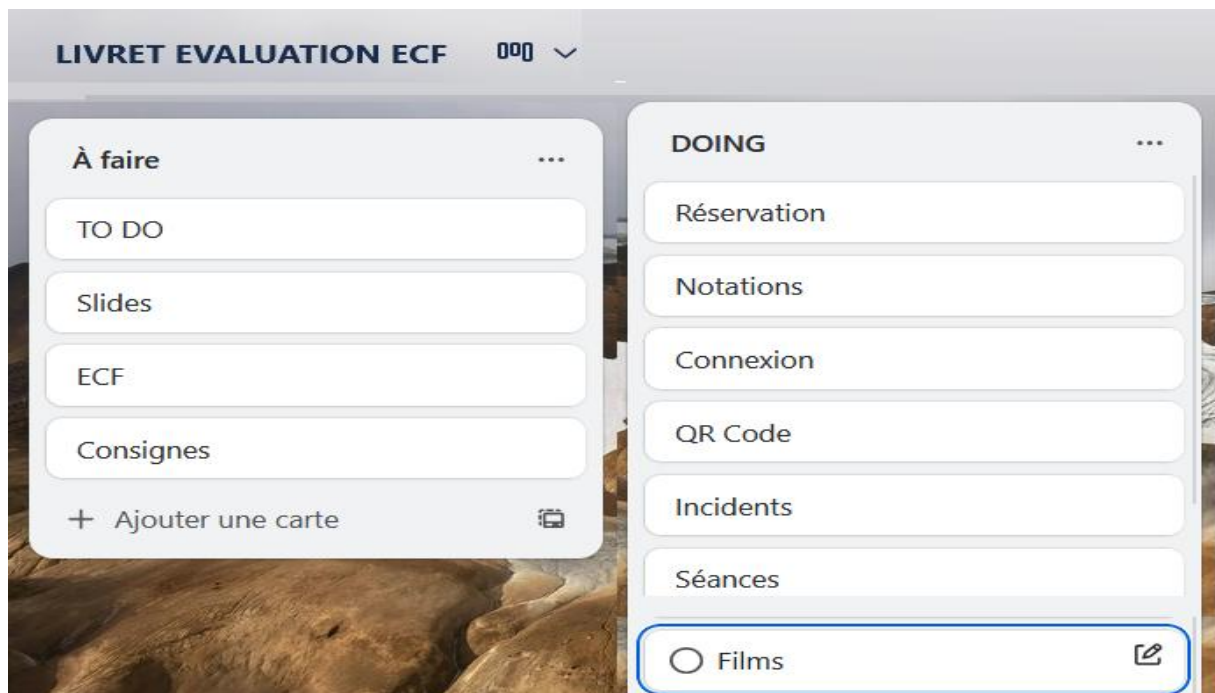
UFR : utilisateur en fauteuil roulant (loi UFR de 2001 : aménagement minimum obligatoire dans les lieux public, tels les cinémas)

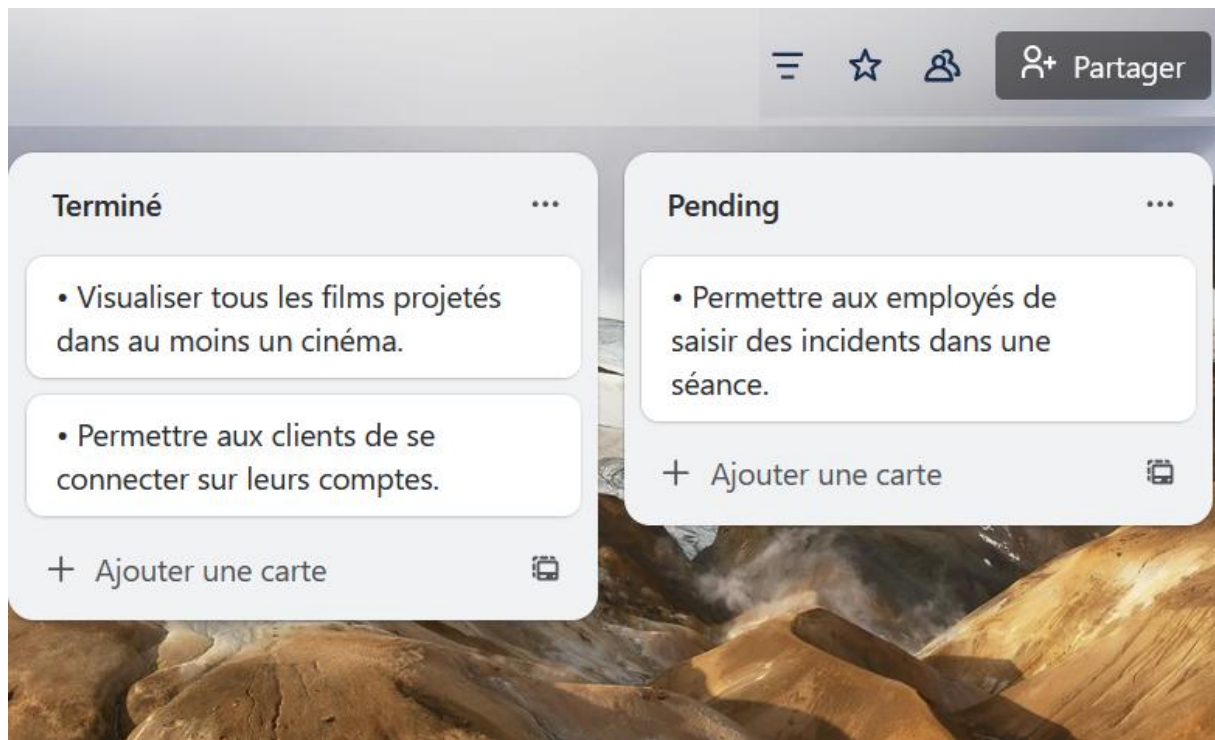
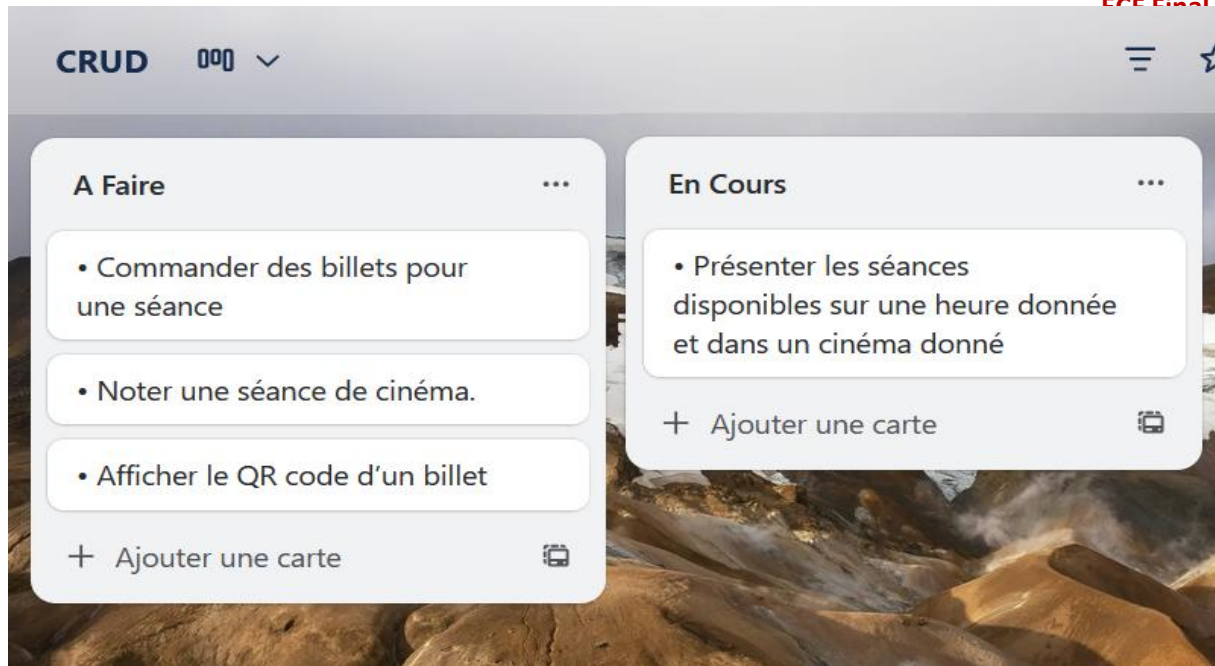
Trello

Figma

MOBILE	MATERIEL	LOGICIEL
	Google Pixel 8a	Android Studio (Meerkat 2024.3.2)
	OS	TECHNOLOGIE(S)
	Android 14	Jetpack Compose + Kotlin
PC	MATERIEL	LOGICIEL
	HP ProBook 650	Wamp Server (3.3.2)
	OS	TECHNOLOGIE(S)
	Windows 11	PHP + MySQL

- On utilise les mêmes outils que l'ECF d'entraînement **Mysql Wamp Phpmyadmin**
- Les nouveautés en outils introduites pour cet ECF Hiver **Jira Bootstrap Android Studio**





TACHES  ▼

TO DO 

FILMS

genre

âge min.

label (coup 2 coeur)

notas

qualités (3D 4K 4DX)

+ Ajouter une carte 

TO DO 

RESERVATIONS

☐ réserver (=hidden) 

prix (=hidden)

- choix des sièges (=hidden)

- sièges > capacités

choix UFR

salles (=hidden)

TACHES  ▼

TO DO 

EMPLOYES

gestion par l'employé

ESPACE CLIENT

voir commande(s)

notas

+ Ajouter une carte 

QR CODE + INCIDENTS

styler nos clients

+ Ajouter une carte 

